

Production AI Agent System: Basketball Multi Agent Assistant

Comprehensive Technical Architecture, Implementation, and Evaluation Report

1. Abstract

The rapid expansion of artificial intelligence and large language models has opened new avenues for solving domain specific knowledge retrieval problems. This project presents a production ready, multi agent AI system designed to navigate and interpret the complex, fragmented rulebooks of four major basketball leagues: the NBA, FIBA, NCAA, and FIBA 3x3. Built upon a distributed microservice architecture, the system combines advanced Retrieval Augmented Generation techniques with real time web search capabilities and deterministic mathematical calculators. By decoupling the architecture into two independent agent systems communicating over a network boundary, the project demonstrates scalable, enterprise grade design. The primary system utilizes LangGraph for state machine orchestration, while the secondary system leverages the Google Agent Development Kit for live data retrieval. Rigorous evaluations using the RAGAS framework and custom retrieval metrics validate the system's efficacy, achieving perfect routing accuracy and exceptional context retrieval scores. This report details the architectural decisions, pipeline implementations, guardrail safety mechanisms (and their associated latency limitations), and a comprehensive failure analysis.

2. Introduction

2.1 Overview of the Problem Domain

Basketball is a globally recognized sport, yet its regulations are far from universal. The four governing bodies operate with distinct rulebooks that collectively span over five hundred pages of dense, legalistic text. The nuances between leagues are critical; for example, a clear path foul in the NBA has different criteria and penalties compared to an unsportsmanlike foul in FIBA. Existing solutions for coaches, referees, and analysts rely on manual

text searches within PDF documents. This approach is highly inefficient and prone to human error, particularly when adjudicating edge cases during high stakes scenarios.

2.2 Project Objectives

The primary objective of this project is to construct an intelligent assistant capable of understanding, retrieving, and reasoning over these divergent rulebooks. The system must not only answer static rules questions but also handle queries requiring live statistics and mathematical projections. Furthermore, the system must adhere to strict production standards, including containerized deployment, session management, input and output safety guardrails, and isolated microservices.

2.3 Scope and Target Audience

The system serves a diverse audience:

- Coaches: For rapid rule clarification during game preparation and live adjustments.
- Referees: To verify complex calls, review edge cases, and ensure cross league consistency.
- Analysts: To provide accurate rule comparisons and statistical context during broadcasts.
- Fans and Students: To settle debates and study the intricacies of basketball officiating.

3. System Architecture and Distributed Design

3.1 High Level Architecture Overview

The system is built as a distributed, containerized microservice ecosystem orchestrated via Docker Compose. It strictly adheres to the principle of separation of concerns, ensuring that distinct functionalities are isolated into specialized services. The architecture is composed of seven independent containers:

1. Agent System A (FastAPI and LangGraph)
2. Agent System B (Google ADK)
3. MCP Suspension Calculator (FastAPI)
4. MCP Performance Calculator (FastAPI)
5. Vector Database (Qdrant)
6. Chat Persistence (MongoDB)
7. Chatbot UI Frontend (Nginx)

3.2 Justification for a Distributed Multi Agent Approach

Production AI systems do not exist in isolation. A monolithic architecture, where all logic resides in a single Python script, is brittle and difficult to scale. By splitting the logic into Agent System A and Agent System B, the project demonstrates the ability to manage network boundaries, handle HTTP timeouts, and integrate disparate technology stacks. System A acts as the core business logic hub, while System B acts as a specialized external service, mirroring real world enterprise architectures where a primary application might call a third party API written by a different team.

3.3 Agent System A: LangGraph Orchestration

Agent System A is the primary application, developed in Python using the LangGraph framework. It exposes a FastAPI POST endpoint for chat queries and utilizes Server Sent Events for streaming responses to the frontend. The core of System A is the Supervisor node.

The Supervisor acts as the intelligent router. It receives the user's query, detects the language, and determines the optimal path through the state machine. The Supervisor is powered by the Qwen 2.5 72B model via OpenRouter. It maintains a state dictionary containing the conversation history, retrieved chunks, tool inputs, and the routing plan.

The LangGraph state machine includes the following nodes:

- Supervisor Node: Determines the route plan and extracts variables (e.g., player names, stat types).
- RAG Agent Node: Handles queries routed for static rulebook knowledge.
- Web Agent Node: Acts as the intermediary for live data. It does not perform the search itself; instead, it formats an HTTP POST request and sends it across the Docker network to Agent System B.
- MCP Suspension Node: Formats parameters and makes an HTTP request to the external Suspension Calculator container.
- MCP Performance Node: Formats parameters and makes an HTTP request to the external Performance Calculator container.
- Human Input Node: Halts execution to request clarification from the user if the target league is ambiguous.

3.4 Agent System B: Live Web Search via Google ADK

Agent System B is an entirely independent microservice running in its own Docker container. It is built using the Google Agent Development Kit and the Gemini 3.6 Flash model. This system is tasked exclusively with fetching live, real time information from the internet, such as current season standings, recent game box scores, and injury updates.

System B exposes a /search endpoint. When it receives a query from System A, the Gemini model uses its native Google Search tool to find the most up to date information. A critical feature of System B is its ability to perform structured data extraction. If the query requires statistical analysis, System B extracts numerical data (points, rebounds, technical fouls) into a strict JSON schema. This structured payload is returned to System A over HTTP, which can then pass the variables directly into the MCP tools.

3.5 Inter System Communication Protocols

The strict requirement for network boundary communication is handled via synchronous HTTP requests. Agent System B is not a Python import within

System A. The Web Agent node in System A utilizes the httpx library to send a JSON payload to `http://system-b:8001/search`. This request is wrapped in a tenacity retry block with exponential backoff and a strict timeout to ensure production level resilience against network failures or downstream latency.

4. Deep Dive: Agent System A Orchestration

To fully appreciate the distributed nature of this multi agent system, a detailed examination of the LangGraph implementation in Agent System A is required. This section provides an in depth technical breakdown of the graph compilation, state management, and the supervisor routing algorithm.

4.1 State Management Schema

In LangGraph, agents do not pass arbitrary data to one another. Instead, they operate on a shared state dictionary. The AgentState schema defines the strictly typed boundaries of this shared memory. The state schema includes:

- `messages`: A chronological list of interaction history, allowing the LLM to maintain conversational context across multiple turns.
- `user_query` and `english_query`: The original input string and its translated counterpart.
- `league_filter`: An optional string variable specifying the target rulebook (NBA, FIBA, NCAA, or FIBA 3x3).
- `next_agent`: A string indicating the next node to execute in the state machine.
- `route_plan`: A list of strings used to manage multi step workflows, allowing the Supervisor to queue sequential agent executions.
- `tool_input`: A dynamic dictionary where extracted parameters (such as player names, game statistics, or foul counts) are stored for downstream consumption by the Model Context Protocol tools.

- `retrieved_chunks` and `image_paths`: Lists storing the dense and sparse search results from the Qdrant vector database, acting as the grounding context for the RAG Agent.
- `mcp_results` and `web_search_results`: Dictionaries that capture the JSON payloads returned from the external HTTP calls to System B and the MCP containers.
- `iteration_count`: An integer counter designed to enforce the infinite loop guardrail.

This strict schema ensures that regardless of which node is currently executing, the data structure remains predictable and robust, preventing silent failures caused by missing keys.

4.2 Graph Compilation and Node Definition

The system architecture defines the workflow as a cyclic state machine. The graph is built using the `StateGraph` class from the `LangGraph` library. The nodes added to the graph include:

- `supervisor`: The entry point and primary decision maker.
- `rag_agent`: Responsible for context synthesis and grounded generation.
- `web_agent`: The HTTP client proxy for calling Agent System B.
- `mcp_suspension` and `mcp_performance`: The HTTP client proxies for calling the external mathematical calculators.
- `human_input`: A blocking node that pauses execution to request clarification.

The execution flow is governed by conditional edges. When the supervisor node completes, a routing function reads the `next_agent` key from the state. If the key corresponds to a valid node, control is passed; otherwise, it defaults to the RAG Agent or terminates. Crucially, the `web_agent` node also contains a conditional edge. After fetching live data from System B, it checks the `route_plan`. If further steps remain, it routes

control back to the supervisor for variable injection and rerouting, enabling powerful chained execution patterns.

4.3 Supervisor Routing Algorithm and Prompt Engineering

The Supervisor node is the most complex component of System A. It operates via a structured prompt engineering strategy using the Qwen 2.5 72B model. The routing process follows these discrete steps:

1. **Language Detection:** The langdetect library analyzes the raw input. If the query is not in English, a preliminary LLM call translates it to ensure optimal performance against the English language vector corpus.
2. **Direct Match Fallback:** If the system is configured in a restricted rag only mode, regex patterns automatically detect the league and bypass the LLM router entirely.
3. **LLM Routing:** In the standard production mode, the translated query and the recent conversation history are fed into the OpenRouter API.
4. **Parameter Extraction:** The LLM is instructed to output a strict JSON schema containing the plan, the league, rewritten sub queries, and extracted tool_input variables.
5. **Re Entry Handling:** If the Supervisor is triggered as a subsequent step (for example, after the Web Agent has fetched a player's box score), it enriches the tool_input dictionary with the newly acquired numerical data before routing to the next node in the route_plan.

The routing prompt explicitly forbids hallucinating variables. If a user asks for a suspension calculation but fails to provide the number of technical fouls, the Supervisor is instructed to output a plan that routes to the Web Agent first, demonstrating advanced logical sequencing.

5. Retrieval Augmented Generation Pipeline

The RAG pipeline within Agent System A is responsible for providing accurate answers based on the official rulebooks. The pipeline was rigorously engineered through experimentation and benchmarking.

5.1 Document Ingestion and Multimodal Parsing

The ingestion phase processes the four official PDF rulebooks. Standard text extraction often destroys the structural integrity of legal documents and ignores visual diagrams. To solve this, the system uses PyMuPDF to parse the PDFs into Markdown format.

Crucially, basketball rulebooks contain vital spatial information in the form of court diagrams and referee signals. During parsing, all images are extracted and saved to disk. A Vision Language Model (Qwen 2.5 VL 72B) analyzes each image and generates a highly detailed, factual textual description of the visual content (e.g., court dimensions, boundary lines). These captions are injected directly into the Markdown text, ensuring the semantic meaning of the diagrams is available to the text based embedding models.

5.2 Chunking Strategy and Justification

Two chunking strategies were implemented and benchmarked: Recursive Character Splitting and Markdown Aware Chunking.

- **Recursive Character Splitting:** This method splits text by character count, using separators like double newlines. While standard, it often breaks articles and sections in half, destroying the legal context.
- **Markdown Aware Chunking:** This method splits the text based on Markdown headers (H1, H2, H3). Because the PyMuPDF parser converts the PDF into structured Markdown, this strategy keeps entire rules and sub articles intact within a single chunk.

Following evaluation, the Markdown Aware strategy (with dynamic sizing based on the length of the specific rule section) was chosen. It ensures that when a rule is retrieved, the entire context of that rule is provided to the generator, preventing partial rule application.

5.3 Embedding Models and Vector Database

The system evaluates two embedding models: BAAI BGE M3 (1024 dimensions) and Sentence Transformers all mpnet base v2 (768 dimensions). The MPNET model was ultimately selected due to its superior performance in capturing the nuanced semantic meaning of sports terminology and legal phrasing, as proven in the evaluation metrics.

The vectors are stored in Qdrant, a high performance vector database running in a dedicated Docker container. Qdrant was chosen for its native support for hybrid search and payload metadata filtering. Each chunk is stored with metadata indicating the source PDF, the league, the page number, and the specific article or section header.

5.4 Retrieval Strategy: Hybrid Search with Cross Encoder Reranking

To maximize retrieval accuracy, the system does not rely solely on dense vector similarity. It implements a Hybrid Search pipeline:

1. Dense Search: Uses the MPNET embeddings to find semantically similar chunks.
2. Sparse Search: Uses BM25 sparse vectors to ensure exact keyword matches (e.g., specific rule numbers, player names).
3. Qdrant combines these scores to return an initial broad set of 15 candidate chunks.

These 15 candidates are then passed to a Cross Encoder Reranking model (BAAI bge reranker v2 m3). Unlike standard embeddings, the cross encoder evaluates the query and the document simultaneously, providing a highly accurate relevance score. The reranker sorts the candidates, and the top 5 chunks are selected for the final context window.

5.5 Grounded Generation and Prompt Engineering

The final step is generation. The Qwen 2.5 72B model is provided with a strict system prompt. The prompt enforces a Chain of Thought process, requiring

the model to generate a <thinking> block to analyze the rules logically before outputting the final answer.

To ensure grounded generation and prevent hallucinations, the prompt explicitly instructs the model to answer ONLY from the retrieved context and to state clearly if the answer is not present. Furthermore, every claim must be supported by an inline citation referencing the specific chunk metadata. The system also supports seamless multilingual interactions, dynamically translating the user's query into English for optimal retrieval, and translating the grounded answer back into the user's native language (French, Spanish, etc.).

6. Deep Dive: Ingestion and Vector Search Mathematics

The quality of a Retrieval Augmented Generation system is entirely dependent on the quality of its underlying data ingestion and vector search algorithms. This section expands on the technical mechanics of the RAG pipeline.

6.1 PyMuPDF and Vision Language Model Captioning

Rulebooks are visually complex documents. The ingestion pipeline begins by utilizing pymupdf4llm to extract the text while preserving markdown formatting. However, text extraction alone discards court diagrams, which contain vital spatial rules (e.g., the exact dimensions of the restricted area). To solve this, the pipeline extracts every image and evaluates it based on file size, pixel dimensions, and aspect ratio to filter out decorative logos. Valid diagrams are encoded into Base64 strings and passed to the Qwen 2.5 VL 72B Vision Language model. The VLM generates a detailed caption describing the visual elements. Using regular expressions, the pipeline locates the image references in the raw markdown and injects these generated captions directly beneath them, creating a truly multimodal text representation.

6.2 Chunking Heuristics

The Markdown Aware chunking strategy dynamically splits the text based on header depth (H1, H2, H3). This ensures that a single chunk contains an entire logical rule. To aid retrieval, the pipeline generates a deterministic chunk ID

using an MD5 hash of the text content combined with a sequential index. Furthermore, a custom regex heuristic scans the first 500 characters of each chunk to identify official legal identifiers (e.g., "Article 34", "Rule No. 5"). This identifier is stored in the chunk's metadata, allowing the final generation model to provide exact rule citations.

6.3 Hybrid Search Mathematics in Qdrant

When a chunk is embedded using the MPNET model, a 768 dimensional dense vector is generated. Simultaneously, the fastembed library generates a sparse vector representation using the BM25 algorithm. Both representations are uploaded to the Qdrant database. During retrieval, Qdrant executes a hybrid search. The dense search uses Cosine Similarity to find semantic matches, which is excellent for queries where the user paraphrases a rule. The sparse search uses term frequency to find exact keyword matches, which is vital for queries mentioning specific proper nouns or rule numbers. Qdrant combines these two scores using Reciprocal Rank Fusion, ensuring that chunks scoring highly in both methods are elevated to the top of the candidate list.

6.4 Cross Encoder Reranking

The initial hybrid search returns 15 candidates. These candidates are often broadly relevant but not perfectly ordered for the specific context of the question. The pipeline employs the BAAI bge reranker v2 m3 model. Unlike a bi encoder (which embeds the query and document separately), a cross encoder concatenates the user's query and the document chunk into a single input string. The model processes this combined input through its self attention layers, allowing it to directly evaluate the relationship between the two texts. This computationally expensive process yields a highly accurate relevance score. The 15 candidates are sorted by this new score, and the top 5 are passed to the generation model.

7. Model Context Protocol Integration

To provide deterministic mathematical reasoning that standard Large Language Models struggle with, the system integrates the Model Context Protocol pattern. This allows the primary agent system to leverage specialized external capabilities.

7.1 Tool 1: Suspension Probability Calculator

The Suspension Probability Calculator is a standalone FastAPI microservice running in its own Docker container. Its purpose is to calculate the likelihood of a player facing a suspension due to technical foul accumulation before the season concludes.

The tool requires the player's current number of technical fouls committed, the number of games played so far, the total games in the season (e.g., 82 for the NBA), the specific league, and the league's foul threshold (e.g., 16 for the NBA). The core logic utilizes a Poisson distribution formula to model the rate of technical fouls per game and projects the cumulative probability over the remaining games. The microservice returns a structured JSON object containing the risk level (Low, Moderate, High, Imminent), the projected game number where the suspension threshold might be breached, and the exact probability percentage. This deterministic approach completely eliminates LLM hallucination for mathematical projections.

7.2 Tool 2: Player Performance Evaluator

The Player Performance Evaluator is the second standalone FastAPI microservice. It provides objective grading of a player's statistical output for a specific game or season.

The tool accepts an extensive payload of box score statistics: points, rebounds, assists, steals, blocks, turnovers, field goals made and attempted, free throws made and attempted, minutes played, and personal fouls. The underlying logic implements a variation of the John Hollinger Game Score formula, weighting positive contributions (points, steals) against negative ones (missed shots, turnovers). The raw score is normalized on a scale from 0 to 100, comparing the player's efficiency and usage against standard league

benchmarks. The final output includes the numerical score, a letter grade ranging from A plus to F, and a brief analytical summary.

8. API Layer and Session Management

The integration and exposure of the system are handled by a robust API layer built with FastAPI, adhering to enterprise deployment standards.

8.1 Endpoints and Server Sent Events

The primary interface for Agent System A is a POST endpoint (/api/chat). To provide a responsive user experience, the system utilizes Server Sent Events for streaming. As the LangGraph state machine progresses through its nodes, the API streams status updates (e.g., "Routing query to agents", "Checking output safety") and the final generated chunks to the frontend in real time, preventing long, silent loading periods.

8.2 MongoDB Session Persistence

Session management is critical for a conversational AI agent. The API layer relies on MongoDB to persist conversation threads. Each request must include a unique thread identifier. The system retrieves the historical context from MongoDB, appending the new user query and the assistant's response, and writes it back to the database. To prevent context window overflow, the system implements a rolling window, passing only the last 20 messages to the LLM during the routing and generation phases.

8.3 End to End Query Lifecycle

To fully understand the distributed architecture, tracking a single query from the user to the system and back provides clarity. The step by step execution pipeline is as follows:

1. **User Request:** The user submits a query through the Chatbot UI. The Nginx reverse proxy routes the HTTP POST request to the System A FastAPI backend (/api/chat).

2. **Security Validation:** The API immediately validates the request. The client's IP address is checked against the RateLimiter (maximum 10 requests per minute). The query is then sanitized (stripping control characters) and checked against the maximum length limit.
3. **Input Guardrail:** The sanitized query is passed to an LLM based topic classifier. If deemed malicious or entirely off topic, execution halts, and a predefined rejection message is returned.
4. **State Initialization:** The system retrieves previous chat history from MongoDB using the thread_id and constructs the initial LangGraph AgentState.
5. **Supervisor Routing:** The LangGraph execution begins at the Supervisor node. The Supervisor analyzes the query, translates it to English if necessary, and determines the route plan (e.g., ["web_agent", "mcp_suspension"]).
6. **Node Execution:** The LangGraph cycles through the nodes specified in the route plan. If the web_agent is called, an HTTP request is made across the Docker network to System B. System B returns a structured JSON payload containing live data, which is injected into the AgentState.
7. **Grounded Generation:** Once all required data is gathered, control passes to the rag_agent or an MCP node to generate the final response.
8. **Output Safety Check:** The generated response is evaluated by the Output Guardrail for Personally Identifiable Information (PII) or harmful content.
9. **Persistence and Return:** The validated response is saved to MongoDB. Finally, the complete payload, including the textual answer, retrieved chunks, image paths, and web results, is returned to the user via the FastAPI endpoint.

9. Guardrails, Safety, and Known Production Limitations

Deploying an AI system requires safety mechanisms to prevent abuse and hallucination. However, the current implementation reveals significant engineering trade offs between security and latency.

9.1 The Problem with LLM Based Topic Guards

Before the query reaches the Supervisor, it passes an Input Guardrail, and before leaving the system, it passes an Output Guardrail. Currently, these guards use LLM calls to classify intent and check for PII or harmful content. While effective at stopping prompt injection or off topic queries, this design is severely flawed for high throughput production. Using heavy language models for basic input output verification introduces unacceptable latency (adding several seconds to the critical path) and inflates API costs unnecessarily. A true production system would replace these LLM guards with faster, localized solutions, such as fine tuned BERT classifiers or strict deterministic rule engines, to handle these checks in milliseconds rather than seconds.

9.2 API Layer Resiliency

Despite the flawed guardrails, the API layer itself implements robust, production ready resiliency. First, IP based rate limiting is enforced via a custom RateLimiter class, capping users at a strict limit (e.g., 10 requests per minute) to prevent denial of service attacks. Second, every incoming query undergoes input sanitization using regular expressions to strip malicious control characters and is checked against a strict MAX_QUERY_LENGTH constant to prevent buffer overflow or context window exhaustion.

To prevent infinite loops within the LangGraph state machine, the system enforces a strict iteration limit on every agent loop. If the graph cycles without reaching a terminal node, it gracefully exits and returns an error message. Furthermore, every internal execution and external network call is wrapped in a request timeout mechanism (e.g., 180 seconds for the ThreadPoolExecutor pipeline, and 5000 milliseconds for MongoDB server selection). Stack traces are suppressed at the API layer, returning clean error messages to the client.

Hardcoded secrets are strictly prohibited; all API keys and database URIs are managed via environment variables defined in a secure configuration file.

10. Evaluation Methodology and Results

The evaluation of the system is the most critical component of the project, demonstrating empirical engineering judgment over random generation.

10.1 Retrieval Metrics and Configuration Comparisons

The retrieval pipeline was subjected to rigorous benchmarking across 16 distinct configurations. The tests compared two embedding models (MPNET and BGE M3), two chunking strategies (Markdown Aware and Recursive), and four retrieval pipelines (Dense, Hybrid, Rerank, Hybrid Rerank).

Across three curated test sets of varying complexity, the configuration of MPNET embeddings with Markdown chunking and a Hybrid Rerank pipeline consistently outperformed all others. In Dataset 2, representing standard rules queries, the chosen configuration achieved:

- Precision at 5: 0.690
- Recall at 5: 1.000
- Hit Rate at 5: 1.000
- Mean Reciprocal Rank: 0.887

By comparison, the standard Dense retrieval using BGE M3 only achieved a Recall of 0.850 and an MRR of 0.704. The inclusion of the Cross Encoder Reranker provided the most significant performance lift, proving the necessity of multi stage retrieval for complex legal texts.

10.2 Generation Evaluation via RAGAS

The generation phase was evaluated using the RAGAS framework, focusing on three core pillars: Faithfulness, Answer Relevance, and Answer Correctness.

In the final production Full Mode (which includes Supervisor query rewriting), the system achieved a Faithfulness score of 0.8285, indicating that over 82

percent of the generated claims were directly traceable to the retrieved context, effectively minimizing hallucination. Context Recall was exceptionally high at 0.8111.

A notable observation was the Answer Relevance score of 0.6922. This lower score is a direct consequence of the Supervisor agent rewriting colloquial user queries into formal search terms; while this improves retrieval recall, it artificially lowers the RAGAS relevance metric which expects the answer to match the exact phrasing of the original query.

10.3 Agent Routing Accuracy

The LangGraph Supervisor was tested against 8 distinct edge cases designed to trigger specific agent combinations. The system achieved a 100 percent success rate in both agent selection (Routing Accuracy) and dynamic league detection. For example, a query regarding "Doncic 14th tech foul" successfully triggered a multi step route: Web Agent to fetch the stats, followed by the MCP Suspension tool.

11. Failure Analysis and System Weaknesses

Despite high evaluation scores, the system is not flawless. Three documented failure cases highlight critical areas for future improvement.

11.1 Failure Case 1: Model Reasoning Failure

Query: "FIBA 3x3: A team is in the penalty. A player on that team commits an offensive foul. Does the other team shoot free throws?" The system incorrectly answered "Yes". Root cause analysis revealed that the retrieval pipeline successfully extracted the exact rule concerning penalty free throws. However, the LLM failed to distinguish the edge case logically. The rule specifies that penalty free throws apply to defensive personal fouls. The model erroneously generalized this to include offensive fouls, demonstrating a pure reasoning failure by the LLM when interpreting compound conditions.

11.2 Failure Case 2: Design Failure in External Tools

Query: "Is Luka Doncic at risk of suspension this season?" When queried during the off season, the MCP Suspension tool returned zeroes for current technical fouls and total games played. The system interpreted this as an active season with perfect behavior, confidently stating there was no suspension risk. The root cause is an architectural design failure: the MCP API lacks a specific status flag to indicate that the requested season data is unavailable or that the league is currently dormant.

11.3 Failure Case 3: Retrieval Failure and Subsequent Hallucination

Query: "NBA: The shot clock is at 0.1 seconds. A player catches an inbound pass and immediately shoots it before the buzzer. It goes in. Does it count?" The system incorrectly asserted that the shot counts. The root cause was a retrieval failure (Context Recall equaled 0.0). The pipeline failed to retrieve the specific Trent Tucker rule, which dictates a minimum of 0.3 seconds is required for a catch and shoot. Lacking the necessary grounding context, the LLM defaulted to its internal pre training regarding general shot clock rules and hallucinated a confident, yet totally incorrect, answer. This highlights the danger of silent retrieval failures.

12. Conclusion and Future Work

The Production AI Agent System successfully demonstrates a robust, scalable architecture for solving complex domain specific problems. By distributing logic across independent containers, implementing sophisticated RAG pipelines with hybrid reranking, and adhering to strict production guardrails, the system serves as a powerful assistant for basketball professionals.

Future iterations will focus on mitigating the identified failure cases. Improvements will include implementing explicit data availability flags in the MCP APIs, expanding the retrieval corpus to include specific casebook interpretations to aid the model in edge case reasoning, and replacing the latent LLM safety guardrails with dedicated fine tuned classifier models to improve overall system throughput.